

K2.pdf

Sanjay Gupta

January 2025

1 Group

Sanjay Gupta — s347gupt — 20885923

Yibo Ma — y439ma — 20883952

2 Gitlab

Please note that in the latest version of K2, the PerformanceTest broke. Please locate the repository at: https://git.uwaterloo.ca/s347gupt/CS452/-/tree/K2_RPS_PERF?ref_type=heads and please use the following hash for testing the RPS Server: 17d43fea1b88cb3cb4221a6b676a4f0528fe0e0c.

For testing the performance task, please use the following https://git.uwaterloo.ca/s347gupt/CS452/-/commits/K2_caching with the following submission hash: fa0d184f0ca502d7f8e2b43791190aafae09ac9c

3 Kernel Structure

3.1 Data Structures

3.1.1 Queues

The code utilizes ring buffers to function as queues (FIFO). Ring buffer is defined using templates (in A0, they were used to pass commands to the Marklin hardware). For the purposes of K1, the ring buffer is used for storing ints (mapping between indices and array of task descriptors). As of K2, the ring buffer template class (deprecated) has been updated to the queue template class which support storing pointers for faster access. In K2, queues are used for managing incoming signup requests in the RPS server.

3.1.2 Priority Queues

Basic implementation (KISS) for min-heap priority queue, used exclusively by the kernel scheduler for determining which task to run. The class creates a static array (of size HEAP.SIZE), (since no heap), and updates a size field on

push/pop operations. Similar to the Ring Buffer class, it is a template but implemented with ints (mapping).

3.2 Classes

The code is organized into classes to enforce encapsulation and reduce coupling between separate processes.

3.2.1 Task Descriptor

Task descriptors isolate logic associated with typical functions needed by any running task; able to do the following:

- Create new tasks (setting themselves as the parent)
- Setting/Returning values associated with tasks (ex: returning TID)
- Each task descriptor contains a context structure: each is 64 bit int: SP, SPSR, ELR, and registers X0-X30. In order to save kernel state on context switches, the address of the context structure used by the kernel is passed into X0 and the task context structure goes to X1.

NOTE: Tasks are only issued to the kernel if memory is available for the task

3.2.2 Memory

Memory for tasks is allocated after the image, at 0x90000. Each stack is allocated 4 MB, and there can be a maximum of 64 tasks. The rationale behind this was that we have approximately 1 GB (minus 76 MB for videocore) after the end of image to work with, and having a task stack of size 256 MB is well within this region.

3.2.3 Kernel

- The kernel has several main private methods, such as Create, MyTid, MyParentTid, Yield, Exit, Send, Receive, Reply. These functions should only be called by the kernel's handler function (aside from bootstrapping in kernel constructor).
- The Kernel uses the memory class for allocating stacks to new tasks (called from create); a ring buffer for popping (create) and pushing (exit) available task ids; a pointer to the active task descriptor (last running task before entering kernel mode); an array of task descriptors (of size MAX_TASKS=64), and a priority queue for O(1) retrieval of the task of highest priority.
- Scheduler: Pops the id of the highest priority task from the ready queue, and returns a pointer to the corresponding entry in the task descriptor table.

- **DispatchTask:** Takes as input a task and sets it's state to active; calls `kernel_to_task_asm` assembly function to perform a context switch. See more details below.
- **Handler:** Takes as input the masked output of the ESR register (parameter 'N' passed in with `svc` call). Based on N, execute privileged kernel operations (retrieving task information), creating new tasks, returning resources, etc.
- **SetRetval:** Save the system call return value into the task context X0 register (`context.x[0]`).

3.3 Assembly

3.3.1 `kernel_to_task_asm`

This is called by the `DispatchTask` function in order to perform a context switch.

- X0 contains address of `kernel_context`
- X1 contains address of `task_context`
- First we store x0 and x1 on the stack, note that `sp` points to the `task_context`
- Store all kernel registers, then store kernel SPSR, SP, ELR.
- Load pointer to `task_context` from stack
- load the registers and processor state
- `eret` to execute task in EL0

3.3.2 Vector Table

Upon issuing SVC calls, the architecture routes the exception to a vector table, defined in `boot.S` (`boot.S` stores the vector table in `VBAR_EL1` system register). Each bank (4 exception handlers, 4 banks) routes to dummy routines refined in `dummy.h`. The one we are interested in is synchronous exceptions from lower EL, AArch64. This is defined in the table to branch to `task_to_kernel_asm`.

3.3.3 `task_to_kernel_asm`

- we load the task context pointer from the stack (taking care not to overwrite X0 and X1 (used in `CREATE` system call)). We essentially perform the opposite of `kernel_to_task_asm`, saving the entire task state, popping the kernel context into X0 and then finally restoring the kernel context.

3.3.4 Cache Invalidation / Cleaning

- As seen later in the document, the cache functionality didn't seem to work (times didn't have much variability).
- We attempted to include code to invalidate the instruction cache and clean and invalidate the entire data cache by looping through each way/set.
- Issued ASM commands enabling the 2 and 12 bits for d/i caches, respectively.

4 UserTask

We define a `usertask` class to represent the abstraction of software running in the user level (above the kernel) in EL0. Based on the input for `PERF` in the `makefile`, `main.cpp` will bootstrap either the `Nameserver` (and `RPS Server`) or perform the performance test. (Set `PERF=0` for `RPS`, `PERF=1` for `Performance Test`).

4.1 Nameserver

- Offers the following utility functions:

- `int WHOIS(const char *name)`: returns `tid` of task with corresponding name
- `int REGISTERAS(const char* name)`: registers the currently running task under the provided name

- When the `FirstUserTask` (`FUT`) runs, it will invoke a system call to create a high priority task that runs the name server - This is constantly in a while loop receive state, so that it immediately gets `RECEIVE_BLOCKED`, only unblocking when other tasks invoke its global functions listed above.

4.2 Rock/Paper/Scissors Server

`RPS Server` is another service that is created by the `FUT`. `RPS` communicates with the name server to register under the name `RPSServer`. Similar to the name server, it indefinitely loops and puts itself in a `RECEIVE_BLOCKED` state. This allows `RPSClients` to communicate commands with it.

- Note that the `RPS` server uses queues to manage creating games with pairs of players.
- **RPSClients**: These are created by the `FUT` for the sole purpose of communicating with the `RPS Server` – the interface is defined so that the `moves` are encoded in a list, and having `QUIT` signals to the `RPS Server` that the registered player wants to quit.

5 RPS Game Test

Note that these sections can be explored by clicking "space" when running the performance task code. Please note that steps have been combined for clarity (need to hit space key to consume each move). Also keep in mind that non-deterministic scheduling may play a role here if following along live.

1. Sign-ups and Initial Pairing

- Player 3 (P3) signs up.
- Player 4 (P4) signs up.
- P3 and P4 are paired together.

2. First Round of RPS

- (move: 0 / 5) - P4's first move
- P4 plays Rock (R).
- Player 5 (P5) signs up.
- (move: 0 / 4) - P3's first move
- P3 plays Rock (R).
- Game Result (GR):
 - P4: Draw
 - P3: Draw
- (move: 1 / 4) - Server updates.

3. New Pairing and Second Round

- P3 plays Rock (R).
- Player 6 (P6) signs up.
- P5 and P6 are paired together.
- (move: 1 / 5) - Updated after pairing.
- P4 plays Paper (P).
- Game Result (GR) (paper covers rock):
 - P4: Win
 - P3: Lose
- (move: 0 / 6) - Updated.
- P5 plays Paper (P).
- (move: 2 / 4) - Server updates.

4. Third Round of RPS

- P3 plays Rock (R).

- (move: 2 / 5) – Server updates.
- P4 plays Scissors (S).
- Game Result (GR):
 - P4: Lose
 - P3: Win
- (move: 0 / 7) – Updated.
- P6 plays Paper (P).
- Game Result (GR):
 - P6: Draw
 - P5: Draw
- (move: 3 / 5) – Updated.

5. Final Round and Player Reclamation

- P4 plays Paper (P)
- (move: 1 / 6) – Updated.
- P5 plays Scissors (S).
- (move: 1 / 7) – Updated.
- P6 plays Scissors (S).
- Game Result (GR):
 - P6: Draw
 - P5: Draw
- (move: 3 / 4) – Updated.

6. Player Reclamation

- Local Player 1 (P3) is marked dead
- P3 and P4 are reclaimed.
- (move: 2 / 6) – Updated.
- P5 plays Scissors (S).
- (move: 2 / 7) – Updated.
- P6 plays Paper (P).
- Game Result (GR):
 - P6: Lose
 - P5: Win
- (move: 3 / 7) – Updated.
- P6 play Paper (P)
- (move: 3 / 6) – Updated.
- P5 quits (marked dead), P6 is reclaimed.

6 Summary

This execution log illustrates the flow of the RPS server, tracking:

- Player sign-ups.
- Game pairings.
- Results of each match.
- Reclaiming of inactive players.

7 Performance Test

- The following methodology was used to measure the timings for the SRR sequences: the sender task (boot) is responsible for spawning the receiver. The order of execution is determined by the priority of the receiver; in our system, boot tasks run at priority **MEDIUM**, so we can have the receiver block before anything is send by spawning it with a **HIGH** priority.
- We keep track of the starting time, and then issue 10000 send requests, which can be issued sequentially and upon the completion of previous requests.
- We then stop the timer in the sender, and get the start time as a reply from the receiver (it has it's own stopwatch); we need to track this in case the receiver starts the transaction first.
- Finally we divide the measured time (using the earlier start time) by 10000 to obtain the average time per each test case.
- NOTE: PerfResults.csv is in the main root folder (outside my_kernel).